

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint)' The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below). Calculate current need of resources by each process. Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:

```
GNU nano 7.2 Banker.c *
#include <stdio.h>

int main()
{
    int i, j, k;
    int n = 5; // Number of processes
    int m = 4; // Number of resources

    int allocation[5][4], max[5][4], need[5][4];
    int available[4];
    int finish[5] = {0};
    int safeSeq[5];
    int count = 0;

    printf("Enter Allocation Matrix:\n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("Enter Max Matrix:\n");
    for(i=0; i<n; i++)
    {
        for(j=0; j<m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter Available Resources:\n");
    for(i=0; i<m; i++)
    {
        scanf("%d", &available[i]);
    }
}
```

```

// Calculate Need Matrix
for(i=0;i<n;i++)
{
    for(j=0;j<m;j++)
    {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

printf("\nNeed Matrix:\n");
for(i=0;i<n;i++)
{
    for(j=0;j<m;j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

// Banker's Algorithm
while(count < n)
{
    int found = 0;

    for(i=0;i<n;i++)
    {
        if(finish[i] == 0)
        {
            int flag = 1;

            for(j=0;j<m;j++)
            {
                if(need[i][j] > available[j])
                {
                    flag = 0;
                    break;
                }
            }

            if(flag == 1)
            {
                for(k=0;k<m;k++)
                {
                    available[k] += allocation[i][k];
                }

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
    {
        printf("\nSystem is NOT in safe state");
        return 0;
    }
}

printf("\nSystem is in SAFE state\n");
printf("Safe Sequence: ");

for(i=0;i<n;i++)
{
    printf("P%d ", safeSeq[i]);
}

return 0;
}

```

OUTPUT:

```
laxmi@laxmi:~  
laxmi@laxmi:~$ nano Banker.c  
laxmi@laxmi:~$ gcc Banker.c  
laxmi@laxmi:~$ ./a.out  
Enter Allocation Matrix:  
0 0 1 4  
0 6 0 2  
0 0 1 2  
1 0 0 0  
1 0 5 4  
Enter Max Matrix:  
0 6 5 6  
0 6 5 2  
0 0 1 2  
1 7 5 0  
2 3 5 6  
Enter Available Resources:  
1 6 2 0  
  
Need Matrix:  
0 6 4 2  
0 0 5 0  
0 0 0 0  
0 7 5 0  
1 3 0 2  
  
System is in SAFE state  
Safe Sequence: P2 P4 P0 P1 P3 laxmi@laxmi:~$ |
```